DÉROULÉ DES SÉANCES

Guide du formateur

Le déroulé détaillé, séance par séance : concepts, démonstrations, pratique guidée, points de vigilance et critères de réussite.

PRÉPARÉ POUR

Usage du formateur

Développer des applications web avec Claude Code · Projet Faya

S1

Prise en main

Objectif de la séance : l'apprenant a Claude Code qui tourne sur le Jarvis Vierge, comprend ce qu'est un terminal et Git à son niveau, a eu sa première vraie conversation, et a rempli son CONTEXT.md + CLAUDE.md.

Pré-requis apprenant : avoir installé Node.js, Git, Claude Code, avoir son abonnement Pro actif. Lui envoyer la liste 2 jours avant.

Déroulé (2 h)

0-10 min — Accueil et cadre

- Expliquer le fil rouge : « dans un mois tu auras une app en ligne que TU auras construite ».
- Poser la règle n°1 : **c'est toi qui tapes**. Moi je montre une fois.
- Montrer le dossier `jarvis-vierge/` : « c'est ton atelier, tout est déjà dedans ».

10-35 min — Concept : terminal + Git (le minimum vital)

- Démo : ouvrir un terminal, `cd`, `ls`, `pwd`. Faire refaire.
- Git en 3 idées : un dépôt = l'historique de ton projet ; `git status` = où j'en suis ; un commit = une photo sauvegardée. Pas plus pour aujourd'hui.
- Démo : `git status` dans le Jarvis Vierge.
- **Piège à montrer** : ne pas avoir peur du terminal, lire ce qui s'affiche. 90% des blocages = on n'a pas lu le message.

35-50 min — Démo : première conversation Claude Code

- Lancer Claude Code dans le dossier.
- Montrer une conversation simple : « explique-moi ce que contient ce workspace ».
- Montrer la différence entre demander (chat) et faire faire (Claude agit, lit des fichiers, propose).

50 min-1 h 50 — Pratique

1. L'apprenant ouvre Claude Code dans `jarvis-vierge/` lui-même.
2. Il copie `.env.example` en `.env` (on laisse vide pour l'instant, on remplira en S9).
3. Il remplit `context/CONTEXT.md` (qui il est, objectifs) en dialoguant avec Claude si besoin.
4. Il édite `CLAUDE.md` sections « Qui je suis » et « Comment tu dois m'aider ».
5. Il lance `/prime` et lit le résumé que Claude fait de lui. On corrige ensemble si Claude a mal compris.
6. Il fait son premier commit : `git add -A` puis `git commit -m "mon contexte"`.

1 h 50-2 h — Récap + devoir

- Récap : terminal OK, Git OK, `/prime` OK, contexte rempli.
- **Devoir** : réfléchir à l'app qu'il veut construire (problème réel, pour qui, 3 fonctions max). L'écrire en 5 lignes.

Points de vigilance formateur

- Ne pas noyer sous Git. 3 commandes suffisent aujourd'hui.
- S'il bloque à l'install, prévoir 20 min de marge (c'est fréquent). Ne pas installer à sa place : le guider.
- **Économie Pro** : lui faire remarquer qu'on est resté sur Sonnet, que la session était courte. Premier réflexe.

Démo possible sur un vrai projet

Montrer rapidement un de tes projets existants avec son `PROGRESS.md` : « voilà où on va, un projet qui se reprend tout seul ». Ne pas détailler, juste donner envie.

Critère de réussite

L'apprenant relance Claude Code seul, tape `/prime`, et Claude le reconnaît via son CONTEXT.md.

S2

La méthode de travail

Objectif : l'apprenant comprend et applique la chaîne **brainstorming → spec → plan → exécution**. Il repart avec son idée d'app cadrée et une spec écrite.

Rappel règle n°1 : c'est lui qui tape.

Déroulé (2 h)

0-10 min — Retour devoir

- Il présente son idée d'app en 5 lignes. On la challenge : faisable en 10 séances ? base de données + login ? Sinon on réduit le périmètre ou on prend une idée de secours (voir programme.md).

10-35 min — Concept : pourquoi ne jamais coder tout de suite

- Le piège du débutant : demander « code-moi une app de X » → Claude part dans tous les sens, on perd des quotas, le résultat ne correspond pas.
- La méthode : d'abord **comprendre** (brainstorming), puis **écrire ce qu'on veut** (spec), puis **découper** (plan), puis **exécuter** étape par étape.
- Analogie : on ne construit pas une maison sans plan. La spec, c'est le plan.
- Démo : invoquer la skill `brainstorming` sur une mini-idée, montrer que Claude pose UNE question à la fois.

35-50 min — Démo : une spec réelle

- Montrer une spec existante d'un de tes projets (structure : objectif, ce qu'on fait, ce qu'on ne fait pas, étapes).
- Insister sur le « hors périmètre » (YAGNI) : dire ce qu'on NE fait PAS est aussi important.

50 min-1 h 50 — Pratique

1. L'apprenant lance le brainstorming sur SON app : « Je veux construire [son idée]. Aide-moi à la cadrer, pose-moi les questions une par une. »
2. Il répond aux questions de Claude. Le formateur veille à ce qu'il réduise le périmètre (3 fonctions max pour commencer).
3. Claude propose 2-3 approches : l'apprenant choisit avec l'aide du formateur (recommander Next.js + Supabase + Vercel, la stack du fil rouge).
4. Claude écrit la spec. L'apprenant la relit et la fait corriger si quelque chose ne colle pas.
5. Commit de la spec.

1 h 50-2 h — Récap + checkpoint + devoir

- Dire « à demain » pour montrer le checkpoint automatique (PROGRESS/NEXT_SESSION).
- **Devoir** : relire sa spec à tête reposée, noter 3 choses qu'il veut ajouter ou retirer.

Points de vigilance

- L'apprenant va vouloir TOUT mettre dans l'app. Le ramener à 3 fonctions. « On ajoutera après, d'abord une base qui marche. »
- Ne pas laisser Claude écrire du code cette séance. On s'arrête à la spec. C'est contre-intuitif pour lui, c'est normal.

- **Économie Pro** : le brainstorming consomme peu (questions-réponses). Lui faire noter : « réfléchir coûte moins cher que coder à l'aveugle ».

Démo possible sur un vrai projet

Montrer comment une de tes specs a évité un refactoring : « si on n'avait pas écrit ça, on aurait codé le mauvais truc ».

Critère de réussite

Une spec claire de son app existe, avec un périmètre réduit et réaliste.

S3

Continuité parfaite

Objectif : l'apprenant maîtrise la continuité (PROGRESS, NEXT_SESSION, `/reprendre`), initialise son projet avec `/nouveau-projet`, et lance le squelette Next.js.

Déroulé (2 h)

0-10 min — Retour devoir

- Il intègre ses 3 ajustements de spec. Rapide.

10-35 min — Concept : ne jamais perdre le fil

- Le vrai problème : une conversation Claude a une limite. Quand elle sature, on perd le contexte. Le débutant recommence tout à chaque fois.
- La solution : **tout ce qui a de la valeur va dans un fichier**, pas dans la conversation.
- Les 4 fichiers d'un projet : CLAUDE.md (constitution), PROGRESS.md (où on en est), DECISIONS.md (pourquoi tel choix), NEXT_SESSION.md (prompt pour repartir).
- `/reprendre` = relire ces fichiers et repartir en 30 secondes dans une conversation neuve.
- Démo : ouvrir un PROGRESS.md réel, montrer comme on comprend l'état du projet en 10 secondes.

35-50 min — Démo : `/nouveau-projet`

- Montrer la séquence : `/nouveau-projet` pose 3 questions (nom, description, stack), crée les 4 fichiers + met à jour INDEX.md, puis propose un plan AVANT de coder.
- Insister : aucun code n'est écrit tant que la structure n'est pas là et le plan validé.

50 min-1 h 50 — Pratique

1. L'apprenant lance `/nouveau-projet` pour son app (à partir de sa spec de S2).
2. Il répond aux 3 questions. Stack : Next.js + Supabase + Vercel.
3. Claude crée `projects/<son-app>/` avec les 4 fichiers. L'apprenant les ouvre et les lit.
4. Claude propose un plan d'attaque. L'apprenant le valide.
5. Première étape du plan : créer le squelette Next.js. Laisser Claude le faire, l'apprenant regarde la sortie et lance `npm run dev` pour voir la page par défaut dans le navigateur.
6. Commit.

1 h 50-2 h — Récap + checkpoint + devoir

- `/reprendre` blanc : fermer, rouvrir, taper `/reprendre`, vérifier que Claude retrouve tout. Effet « waouh » garanti.
- **Devoir** : ouvrir une NOUVELLE conversation demain et reprendre son projet seul avec `/reprendre`. Noter si ça marche.

Points de vigilance

- C'est LA séance qui change tout pour lui. Prendre le temps sur le « pourquoi ».
- S'assurer que `npm run dev` tourne (port occupé, Node manquant → dépanner).

- **Économie Pro** : marteler que `/reprendre` dans une conversation neuve coûte BEAUCOUP moins qu'une conversation de 3 h qui traîne. C'est le réflexe-clé de l'abonnement Pro.

Démo possible sur un vrai projet

Montrer ton INDEX.md avec plusieurs projets : « je peux jongler entre eux sans rien perdre ».

Critère de réussite

L'apprenant ferme tout, rouvre, tape `/reprendre` et son projet repart intact.

S4

Lire et piloter du code

Objectif : l'apprenant sait écrire un prompt efficace, lire ce que Claude produit, repérer un problème et faire corriger. Résultat : une première vraie page de son app en local.

Déroulé (2 h)

0-10 min — Retour devoir

- A-t-il réussi `/reprendre` seul hier ? Débriefer les éventuels ratés (souvent : mauvais dossier).

10-35 min — Concept : le bon prompt

- Un prompt flou = un résultat flou. Les 4 ingrédients : le **contexte** (sur quoi), l'**objectif** (quoi), les **contraintes** (comment/avec quoi), le **résultat attendu** (à quoi ça ressemble si c'est réussi).
- Montrer un prompt vague vs un prompt précis sur la même tâche, comparer les résultats.
- Apprendre à lire la sortie : Claude dit ce qu'il fait (lit tel fichier, modifie tel autre). Ne pas valider en aveugle.
- Règle : si tu ne comprends pas ce que Claude a fait, demande-lui d'expliquer avant d'accepter.

35-50 min — Démo : itérer sur une page

- Démo : « crée une page d'accueil avec un titre et un bouton ». Puis itérer : « le bouton doit être centré », « ajoute un sous-titre ». Montrer qu'on construit par petites touches.
- Montrer comment pointer une erreur : copier le message d'erreur du terminal/navigateur et le donner à Claude.

50 min-1 h 50 — Pratique

1. L'apprenant prend la prochaine étape de son plan (une page concrète de son app, ex. la page principale).
2. Il écrit le prompt avec les 4 ingrédients. Le formateur relit le prompt AVANT qu'il l'envoie.
3. Claude code. L'apprenant lit la sortie, lance `npm run dev`, regarde dans le navigateur.
4. Il itère 2-3 fois pour ajuster (texte, disposition).
5. Si erreur : il copie le message et le donne à Claude. Apprentissage du cycle erreur → correction.
6. PROGRESS.md mis à jour, commit.

1 h 50-2 h — Récap + checkpoint + devoir

- Récap : prompt en 4 ingrédients, lire la sortie, cycle d'erreur.
- **Devoir** : ajouter seul une petite page ou un élément à son app, en écrivant un bon prompt. Noter une question s'il bloque.

Points de vigilance

- Il va vouloir accepter tout ce que Claude propose sans lire. Le freiner : « qu'est-ce que Claude vient de changer ? ».
- Ne pas le laisser empiler 10 demandes d'un coup. Une touche à la fois, on vérifie, on continue.
- **Économie Pro** : itérer par petites touches claires coûte moins qu'un gros prompt confus qu'on refait 3 fois.

Démo possible sur un vrai projet

Montrer un bug réel que tu as corrigé en donnant le message d'erreur à Claude : dédramatiser l'erreur.

Critère de réussite

Une page de son app s'affiche dans le navigateur, et l'apprenant a corrigé au moins une erreur par lui-même.

S5

C'est quoi tout ça ? (le vocabulaire)

Objectif : l'apprenant comprend enfin les mots qu'il voit partout : MCP, agent, skill, plugin, hook, commande. Sans jargon, avec des analogies et une démo de chacun.

Note : séance plus « théorique » mais ancrée. Chaque notion = une démo vivante sur le Jarvis Vierge.

Déroulé (2 h)

0-10 min — Retour devoir

- Il montre la page ajoutée seul. On débriefe son prompt.

10-50 min — Concept + démo de chaque brique

Utiliser une analogie filée : **Claude Code = un employé très doué**. Les briques sont ses outils et ses règles.

- **Commande** (`/prime`, `/reprendre`) : un raccourci que je tape. Comme une macro. Démo : `/prime`.
- **Skill** : une méthode de travail que Claude sait appliquer (ex. brainstorming, design). Comme un mode d'emploi qu'il sort quand il en a besoin. Démo : lister les skills du dossier.
- **Agent (sous-agent)** : un collègue spécialiste à qui Claude délègue une tâche (ex. relire le code, auditer la sécurité). Démo : montrer la liste des agents du Jarvis.
- **MCP** : une prise qui branche Claude sur un service extérieur (base de données, GitHub, navigateur). Comme un câble USB vers le monde réel. Démo : ouvrir `.mcp.json`, expliquer les prises.
- **Hook** : une règle automatique déclenchée à un moment (ex. « bloquer si quelqu'un essaie de committer une clé »). Comme un garde-fou. Démo : montrer `.claude/hooks/`.
- **Plugin** : un pack qui apporte d'un coup des skills/agents/commandes/MCP. Comme installer une extension. Démo : ouvrir `PLUGINS.md`.

Tableau récap à lui faire noter (voir mémo S5).

50 min-1 h 50 — Pratique

1. L'apprenant explore SON Jarvis : il liste ses agents, ouvre une skill, lit un hook, regarde `.mcp.json`.
2. Il demande à Claude : « quels agents j'ai pour m'aider sur du frontend ? et pour la sécurité ? » et lit les réponses.
3. Il déclenche un agent simple sur son app (ex. code-reviewer sur un fichier existant) juste pour voir le mécanisme de délégation.
4. Il note dans son mémo, avec ses mots, ce que fait chaque brique.

1 h 50-2 h — Récap + checkpoint + devoir

- Quiz express oral : « c'est quoi un MCP ? un hook ? ».
- **Devoir** : écrire dans son mémo une phrase perso pour chaque brique.

Points de vigilance

- Ne pas être exhaustif ni technique. Objectif = qu'il ne soit plus perdu par le vocabulaire.
- Rester sur les analogies. S'il veut creuser, renvoyer aux séances suivantes (MCP en S9, hooks en S14, agents en S13).

- **Économie Pro** : déclencher un agent consomme plus (c'est une 2e conversation). À utiliser quand ça vaut le coup, pas pour tout.

Critère de réussite

L'apprenant explique les 6 briques avec ses propres mots.

S6

Design avec Claude Design, puis code

Objectif : l'apprenant maquette l'interface de son app dans **Claude Design** (le design vient en premier), puis la transpose en code (Next.js + Tailwind) dans Claude Code. Il apprend à décrire un visuel et à faire un handoff design → code propre.

Méthode clé de cette séance : on ne code PAS l'UI à l'aveugle. On la dessine d'abord dans Claude Design, on valide le visuel, ENSUITE on le transpose en code.

Déroulé (2 h)

0-10 min — Retour devoir

- Il relit ses phrases sur les 6 briques. Corriger les contresens.

10-35 min — Concept : pourquoi designer avant de coder

- Le piège : demander du code UI directement → on itère visuellement DANS le code, c'est lent et coûteux en quota.
- La bonne méthode : **Claude Design d'abord**. On obtient une maquette qu'on regarde, qu'on ajuste vite, avant d'écrire une ligne de code.
- Une fois le visuel validé, Claude Code le **transpose** en Next.js + Tailwind fidèlement.
- Décrire un design à Claude : ambiance (3 adjectifs), référence (« comme Stripe », « épuré »), couleurs, disposition. Claude Design ne devine pas le goût.
- Les skills du Jarvis qui aident à la transposition fidèle : `frontend-design`, `ui-design-system`, `tailwind-patterns`, `theme-factory`.

35-50 min — Démo : le cycle design → code

- Démo : décrire la page d'accueil, la faire maquetter dans Claude Design, ajuster (couleur, disposition), puis demander à Claude Code de la transposer en page Next.js + Tailwind. Montrer que le code colle à la maquette.

50 min-1 h 50 — Pratique

1. L'apprenant choisit l'ambiance de son app (3 adjectifs + éventuelle référence).
2. Il **maquette dans Claude Design** sa page principale. Il itère sur le visuel (couleurs, titres, boutons, espacements) AVANT de coder.
3. Quand le visuel lui plaît, il demande à Claude Code de **transposer** la maquette en code Next.js + Tailwind (en s'appuyant sur les skills de design).
4. Il compare le rendu codé à la maquette dans le navigateur, ajuste les écarts.
5. Il fixe une cohérence (mêmes couleurs/boutons) pour les pages suivantes.
6. PROGRESS.md + commit.

1 h 50-2 h — Récap + checkpoint + devoir

- **Devoir** : maquetter seul une page secondaire dans Claude Design (même ambiance), sans encore la coder.

Points de vigilance

- Bien faire respecter l'ordre : **design d'abord, code ensuite**. C'est contre-intuitif pour un débutant qui veut « voir du code ».
- Éviter de changer d'ambiance à chaque page. Fixer une direction visuelle et s'y tenir.
- Rendu lourd (3D, effets) : sa machine peut ne pas avoir de GPU. Rester sur du 2D propre.
- **Économie Pro** : itérer le visuel dans Claude Design coûte moins que de refaire du code 5 fois. C'est tout l'intérêt du design-first.

Démo possible sur un vrai projet

Montrer une maquette Claude Design d'un de tes projets et l'écran codé correspondant : prouver que le handoff marche.

Critère de réussite

L'apprenant a une maquette validée dans Claude Design ET sa transposition codée fidèle, pour sa page principale.

S7

Responsive et accessibilité

Objectif : l'app est utilisable sur mobile et accessible (contrastes, navigation clavier, labels). L'apprenant découvre l'agent `web-accessibility-checker`.

Déroulé (2 h)

0-10 min — Retour devoir

- Vérifier la page secondaire stylisée. Cohérence visuelle OK ?

10-35 min — Concept : mobile d'abord + accessibilité

- La majorité des utilisateurs sont sur mobile (pertinent en Côte d'Ivoire : le mobile domine). Une app qui casse sur petit écran = inutilisable.
- Responsive : Claude gère avec les classes Tailwind (`sm:`, `md:`, `lg:`). Je décris les écrans cibles.
- Accessibilité (a11y) : contrastes suffisants, textes alternatifs, navigation au clavier, labels de formulaire. C'est de la qualité ET ça évite d'exclure des utilisateurs.
- Démo : réduire la fenêtre du navigateur, montrer ce qui casse, demander un correctif responsive.

35-50 min — Démo : l'agent accessibilité

- Lancer l'agent `web-accessibility-checker` (ou la skill `accessibility`) sur une page, lire ses recommandations, en appliquer une ou deux.

50 min-1 h 50 — Pratique

1. L'apprenant teste son app en mode mobile (DevTools du navigateur, vue responsive).
2. Il demande à Claude de corriger ce qui casse sur petit écran.
3. Il fait auditer une page clé par l'agent accessibilité.
4. Il applique les corrections prioritaires (contraste, labels, focus clavier).
5. PROGRESS.md + commit.

1 h 50-2 h — Récap + checkpoint + devoir

- **Devoir** : tester toute l'app sur son propre téléphone (ou en vue mobile) et noter 3 problèmes.

Points de vigilance

- Ne pas viser la perfection a11y, viser les gains évidents (contraste, clavier, labels).
- Lui montrer comment ouvrir la vue responsive des DevTools (F12).
- **Économie Pro** : faire auditer par un agent est utile mais coûte ; grouper l'audit sur les pages importantes.

Critère de réussite

L'app ne casse pas en vue mobile et a passé un audit a11y avec corrections appliquées.

S8

Commandes et skills perso

Objectif : l'apprenant comprend qu'il peut fabriquer ses propres outils. Il crée une commande perso simple et sait invoquer une skill à la demande.

Déroulé (2 h)

0-10 min — Retour devoir

- Les 3 problèmes mobiles relevés : on en corrige un ou deux en ouverture.

10-35 min — Concept : fabriquer ses propres raccourcis

- Une commande = un fichier `.md` dans `.claude/commands/`. Son contenu est l'instruction que Claude exécute quand je tape `/nom`.
- Exemples utiles pour lui : `/point` (résumé de l'état de l'app), `/todo` (liste ce qu'il reste à faire), `/explique` (explique un fichier en français simple).
- Une skill s'invoque quand on veut appliquer une méthode précise ; une commande, c'est MON raccourci personnel.
- Démo : ouvrir une commande existante (`prime.md`) pour montrer la structure, puis en créer une nouvelle ensemble.

35-50 min — Démo : créer `/point`

- Créer `.claude/commands/point.md` avec une instruction simple (« Résume l'état actuel de mon app en lisant PROGRESS.md et donne-moi les 3 prochaines actions »). Tester `/point`.

50 min-1 h 50 — Pratique

1. L'apprenant crée 1 à 2 commandes utiles pour SON app (ex. `/point`, `/explique`).
2. Il les teste.
3. Il invoque une skill à la demande sur une tâche (ex. systematic-debugging si un bug traîne, ou frontend-design pour une page).
4. Pendant qu'il y est : finir les corrections responsive/a11y de S7 restantes.
5. PROGRESS.md + commit.

1 h 50-2 h — Récap + checkpoint + devoir

- Mi-parcours : rappeler qu'on a fini la moitié des séances, l'app prend forme côté interface. Semaine prochaine : on branche la vraie base de données.
- **Devoir** : imaginer une 3e commande qui lui faciliterait la vie et l'écrire.

Points de vigilance

- Garder les commandes simples. Une commande = une intention claire.
- Ne pas confondre commande (son raccourci perso) et skill (méthode fournie).
- **Économie Pro** : une bonne commande standardise ses prompts → moins d'allers-retours, moins de quota gaspillé.

Démo possible sur un vrai projet

Montrer une de tes commandes perso et expliquer le temps qu'elle te fait gagner.

Critère de réussite

L'apprenant a créé au moins une commande perso fonctionnelle et l'utilise.

S9

MCP en pratique

Objectif : l'apprenant branche son premier MCP réel (Supabase) et le navigateur, comprend le rôle du `.env` et du coffre à secrets. À la fin, Claude peut parler à une vraie base de données.

Pré-requis : l'apprenant crée un compte Supabase (gratuit) AVANT la séance, ou on le fait au début.

Déroulé (2 h)

0-10 min — Retour devoir

- Sa 3e commande. Rapide.

10-35 min — Concept : brancher Claude sur le monde réel

- Rappel S5 : un MCP = une prise. Aujourd'hui on branche vraiment.
- Supabase = une base de données PostgreSQL hébergée + auth, gratuite pour démarrer. Parfait pour son app.
- Les clés ne se collent JAMAIS dans le chat ni en dur dans le code. Elles vivent dans `.env` (ignoré par git) et dans le coffre `secrets/`.
- Workflow : `/ajouter-cle` pour stocker, `/generer-env` pour injecter dans un projet.
- Démo : montrer `.mcp.json` (les prises à placeholders) et `.env.example`.

35-50 min — Démo : remplir le .env et vérifier la connexion

- Montrer où récupérer le token Supabase (Dashboard > Account > Access Tokens) et le project-ref.
- Le mettre dans `.env`, relancer Claude Code, vérifier que le MCP Supabase se connecte (`/mcp`).

50 min-1 h 50 — Pratique

1. L'apprenant crée son projet Supabase (si pas fait).
2. Il stocke son token via `/ajouter-cle` (le formateur rappelle : jamais en clair dans le chat).
3. Il remplit son `.env` (Supabase + éventuellement GitHub).
4. Il relance Claude Code, tape `/mcp`, vérifie que Supabase est connecté.
5. Il demande à Claude : « liste les tables de ma base » (vide pour l'instant, c'est normal). Le lien est prouvé.
6. Il branche aussi le MCP navigateur (chrome-devtools) et demande à Claude d'ouvrir son app locale.
7. PROGRESS.md + commit (le `.env` ne sera PAS commité, montrer qu'il est gitignored).

1 h 50-2 h — Récap + checkpoint + devoir

- **Devoir** : lister (sur papier) les 2-3 tables dont son app aura besoin (ex. users, tâches, projets).

Points de vigilance

- **Sécurité clés** : c'est LE moment où il peut faire une bêtise. Marteler : jamais de clé dans le chat, jamais en dur. Le hook secret-scanner le protège mais la discipline d'abord.
- Supabase gratuit : se met en pause après 7 jours d'inactivité (DNS qui échoue). Le prévenir : si « fetch failed », réveiller le projet dans le dashboard.
- **Économie Pro** : brancher un MCP ne coûte pas de quota en soi ; les requêtes via MCP, un peu. Rester sobre.

Démo possible sur un vrai projet

Montrer un de tes projets branché à Supabase qui lit/écrit des données réelles.

Critère de réussite

`/mcp` montre Supabase connecté, et Claude peut interroger la base.

S10

Base de données

Objectif : l'app lit et écrit de vraies données. L'apprenant comprend un schéma simple (tables, colonnes, relations) et fait un CRUD de base.

Déroulé (2 h)

0-10 min — Retour devoir

- Il montre ses 2-3 tables imaginées. On les affine ensemble.

10-35 min — Concept : une base de données sans jargon

- Une **table** = un tableur (lignes = enregistrements, colonnes = champs).
- Une **relation** = un lien entre tables (une tâche appartient à un utilisateur).
- **CRUD** = les 4 actions : Créer, Lire, Mettre à jour, Supprimer. 90% des apps, c'est ça.
- La skill `design-database-schema` (commande `/design-database-schema`) aide à concevoir le schéma proprement.
- Démo : concevoir une table simple et l'expliquer.

35-50 min — Démo : créer une table + une lecture

- Faire créer une table dans Supabase (via la commande de schéma ou le dashboard), puis une page qui liste ses lignes.
- Montrer la boucle : donnée en base → affichée dans l'app.

50 min-1 h 50 — Pratique

1. L'apprenant conçoit son schéma (2-3 tables) avec l'aide de Claude.
2. Il crée les tables dans Supabase.
3. Il implémente la **lecture** : une page qui affiche les données réelles.
4. Il implémente la **création** : un formulaire qui ajoute une ligne.
5. Il teste le cycle complet : j'ajoute → ça s'affiche.
6. PROGRESS.md + DECISIONS.md (noter le choix du schéma) + commit.

1 h 50-2 h — Récap + checkpoint + devoir

- **Devoir** : ajouter la mise à jour OU la suppression (le U ou le D du CRUD) sur une entité, seul.

Points de vigilance

- Garder le schéma minimal (pas de sur-conception). On enrichit si besoin.
- Attention au plafond Supabase gratuit : les lectures peuvent être tronquées à 1000 lignes silencieusement. Pas bloquant à son échelle, mais bon à savoir.
- Bien noter le schéma dans DECISIONS.md : c'est une décision structurante.
- **Économie Pro** : faire concevoir le schéma d'un coup (pas table par table sur 10 messages).

Démo possible sur un vrai projet

Montrer une table réelle d'un de tes projets et l'app qui l'utilise.

Critère de réussite

L'apprenant ajoute une donnée via son app et la voit s'afficher, depuis une vraie base Supabase.

S11

Authentification

Objectif : l'app a des comptes utilisateurs. L'apprenant comprend login/session et protège les pages réservées.

Déroulé (2 h)

0-10 min — Retour devoir

- Son U ou D du CRUD. Débrief rapide.

10-35 min — Concept : qui a le droit de faire quoi

- **Authentification** = prouver qui je suis (login). **Autorisation** = ce que j'ai le droit de faire.
- Supabase fournit l'auth clé en main (email/mot de passe, et plus). Pas besoin de réinventer.
- Notion de **session** : une fois connecté, l'app se souvient de moi.
- **Routes protégées** : certaines pages ne doivent être vues que connecté.
- Démo : expliquer le parcours inscription → connexion → page privée → déconnexion.

35-50 min — Démo : brancher l'auth Supabase

- Montrer la mise en place de l'inscription + connexion via Supabase Auth, et une page protégée qui redirige si non connecté.

50 min-1 h 50 — Pratique

1. L'apprenant active l'auth Supabase (email/mot de passe).
2. Il crée les pages inscription + connexion.
3. Il protège au moins une page (visible seulement connecté).
4. Il relie les données à l'utilisateur (ex. « mes tâches » = celles de l'utilisateur connecté).
5. Il teste : créer un compte, se connecter, voir ses données, se déconnecter.
6. PROGRESS.md + DECISIONS.md + commit.

1 h 50-2 h — Récap + checkpoint + devoir

- Rappeler qu'on a maintenant une app complète : interface + base + comptes. Il reste à fiabiliser et déployer.
- **Devoir** : tester les cas limites (mauvais mot de passe, email déjà pris) et noter le comportement.

Points de vigilance

- L'auth est la zone la plus sensible côté sécurité. Introduire l'idée qu'on la testera en S16.
- Bien séparer données publiques / privées. Introduire (sans approfondir) les règles d'accès Supabase (RLS) : « chaque utilisateur ne voit que ses données ».
- **Économie Pro** : l'auth est un bloc connu, un prompt bien cadré suffit souvent. Éviter de tâtonner.

Démo possible sur un vrai projet

Montrer un de tes projets avec login et données par utilisateur.

Critère de réussite

L'apprenant crée un compte, se connecte, et ne voit que ses propres données.

S12

Économie Pro et continuité avancée

Objectif : l'apprenant sait tenir ses quotas Pro sur la durée et choisir entre prolonger une session ou en ouvrir une neuve. Séance plus légère côté code, on en profite pour consolider.

Déroulé (2 h)

0-10 min — Retour devoir

- Les cas limites de l'auth. On corrige un comportement gênant si besoin.

10-40 min — Concept : faire durer son abonnement Pro

- Pourquoi c'est vital : Pro a des quotas serrés, un mois de formation intensif peut les épuiser.
- Les leviers :
- **Modèle** : rester sur Sonnet par défaut, réserver les modèles plus lourds aux moments durs.
- `/clear` entre deux tâches sans rapport : repartir propre, ne pas traîner tout l'historique.
- **Sessions courtes et ciblées** : une tâche = une session.
- **Continuité par fichiers** : PROGRESS/NEXT_SESSION pour reprendre neuf plutôt que d'allonger.
- **Prompts précis** (les 4 ingrédients de S4) : moins d'allers-retours.
- **Agents avec parcimonie** : chaque agent = une conversation en plus.
- Signal d'alerte : le checkpoint auto à ~70% (le Jarvis le fait tout seul). Lui montrer à quoi ça ressemble.

40-55 min — Démo : conversation longue vs neuve

- Montrer concrètement : une conversation qui traîne devient lente et chère ; `/reprendre` dans une neuve est net et rapide.

55 min-1 h 50 — Pratique (consolidation)

1. L'apprenant fait le point sur son app : qu'est-ce qui marche, qu'est-ce qui reste ?
2. Il met à jour proprement PROGRESS.md et NEXT_SESSION.md lui-même.
3. Il règle 1-2 petits problèmes en suspens, en pratiquant `/clear` entre eux.
4. Il vérifie son INDEX.md.

1 h 50-2 h — Récap + checkpoint + devoir

- **Devoir** : tenir un mini-journal de sa consommation sur 2 jours (quand ça consomme le plus) et le rapporter.

Points de vigilance

- Ne pas culpabiliser sur le quota : donner des réflexes, pas de la peur.
- Vérifier qu'il sait vraiment faire `/clear` et `/reprendre` au bon moment.
- Profiter de cette séance-tampon pour rattraper un éventuel retard sur l'app.

Démo possible sur un vrai projet

Montrer comment tu enchaînes des sessions neuves avec `/reprendre` sur un gros projet.

Critère de réussite

L'apprenant explique 3 leviers d'économie et les applique sans qu'on le lui rappelle.

S13

Agents et revue de code

Objectif : l'apprenant fait relire son app par des agents spécialisés et corrige les problèmes avec la méthode de debug systématique.

Déroulé (2 h)

0-10 min — Retour devoir

- Son journal de consommation. Un conseil ciblé selon ses pics.

10-35 min — Concept : déléguer à des spécialistes

- Rappel S5 : un agent = un collègue expert. Aujourd'hui on les met au travail.
- `code-reviewer` : relit le code, repère bugs, sécurité, qualité.
- `debugger` + skill `systematic-debugging` : face à un bug, on ne tâtonne pas. On reproduit, on formule une hypothèse, on teste, on isole.
- La commande `/code-review` fait une revue cadrée.
- Démo : lancer une revue sur un fichier de son app, lire les retours, trier l'important.

35-50 min — Démo : debug systématique

- Prendre un vrai bug (ou en provoquer un). Montrer la démarche : reproduire → hypothèse → test → correction → vérification. Pas de « essaie ça au hasard ».

50 min-1 h 50 — Pratique

1. L'apprenant lance `/code-review` (ou l'agent code-reviewer) sur son app.
2. Il lit les retours, classe par priorité (bugs réels d'abord, détails ensuite).
3. Il corrige les problèmes importants, un par un, en vérifiant à chaque fois.
4. S'il a un bug tenace : appliquer `systematic-debugging` ensemble.
5. PROGRESS.md + commit.

1 h 50-2 h — Récap + checkpoint + devoir

- **Devoir** : relancer une revue seul après ses corrections et vérifier que les points sont réglés.

Points de vigilance

- Un agent peut signaler beaucoup de choses : apprendre à TRIER, pas à tout corriger aveuglément.
- Ne pas accepter une « correction » sans comprendre ce qu'elle change.
- **Économie Pro** : une revue = une grosse conversation. La lancer sur des morceaux ciblés, pas tout le projet à chaque fois.

Démo possible sur un vrai projet

Montrer un vrai retour de code-review sur un de tes projets et comment tu as priorisé.

Critère de réussite

L'app a été revue et les problèmes prioritaires sont corrigés et vérifiés.

S14

Hooks et garde-fous

Objectif : l'apprenant comprend les hooks déjà actifs dans son Jarvis et voit comment ils le protègent. Il vérifie que .env est protégé, que le formatage est automatique, et que committer une clé est impossible.

Déroulé (2 h)

0-10 min — Retour devoir

- Sa revue de contrôle. Les points sont-ils bien réglés ?

10-35 min — Concept : des règles automatiques

- Rappel S5 : un hook = une règle déclenchée automatiquement à un moment.
- Les hooks déjà installés dans son Jarvis :
- **dangerous-command-blocker** : bloque les commandes destructrices (ex. effacer tout).
- **secret-scanner** : empêche de committer une clé/token.
- **blocage .env** en écriture : protège ses secrets.
- **formatage auto** (prettier/black) après édition : code propre sans y penser.
- **scope-guard** : prévient quand on modifie des fichiers hors du périmètre prévu.
- Pourquoi c'est précieux pour un débutant : la machine rattrape les erreurs classiques.
- Démo : tenter de committer un faux fichier contenant une clé bidon → montrer le blocage.

35-50 min — Démo : formatage auto + commande dangereuse

- Éditer un fichier mal indenté, montrer qu'il est reformaté tout seul.
- Tenter une commande bloquée (ex. une suppression large) → montrer le refus.

50 min-1 h 50 — Pratique

1. L'apprenant déclenche volontairement chaque garde-fou pour voir l'effet (clé bidon, commande à risque).
2. Il vérifie que son `.env` réel est bien gitignored (`git status` ne doit pas le lister).
3. Il regarde le contenu des hooks dans `.claude/hooks/` et lit le `settings.json` (sans tout comprendre, juste voir le lien).
4. Optionnel : introduire `hookify` pour créer une règle simple s'il en a l'envie.
5. PROGRESS.md + commit (propre, formaté automatiquement).

1 h 50-2 h — Récap + checkpoint + devoir

- **Devoir** : relire son historique git et vérifier qu'aucune clé n'a jamais été commitée.

Points de vigilance

- Les hooks peuvent parfois bloquer une commande légitime (faux positif). Lui apprendre à reformuler plutôt que désactiver.
- Insister : les hooks protègent, mais la discipline (clés dans .env, jamais dans le chat) reste la base.
- **Économie Pro** : les hooks sont locaux, ils ne consomment pas de quota Claude.

Critère de réussite

L'apprenant a vu chaque garde-fou agir et confirme qu'aucune clé n'est dans son historique git.

S15

Déploiement

Objectif : l'app est EN LIGNE sur Vercel, avec ses variables d'environnement en production. L'apprenant comprend previews et déploiement.

Pré-requis : compte Vercel (gratuit) + code sur un repo GitHub.

Déroulé (2 h)

0-10 min — Retour devoir

- Historique git vérifié, aucune clé. Si une clé traîne : on traite en priorité (régénérer la clé chez le fournisseur).

10-35 min — Concept : passer du local au monde

- En local, seul moi vois l'app (`npm run dev`). Déployer = la rendre accessible par une URL publique.
- **Vercel** : hébergement fait pour Next.js, gratuit pour démarrer, se connecte au repo GitHub.
- **Variables d'environnement en prod** : mes clés (.env local) doivent être recopiées dans Vercel, PAS commitées. L'app en prod lit celles de Vercel.
- **Preview** : chaque changement crée une URL de test avant la prod.
- Démo : montrer un projet déployé et son dashboard Vercel.

35-50 min — Démo : premier déploiement

- Pousser le code sur GitHub, importer le projet dans Vercel, configurer les variables d'env, lancer le déploiement, ouvrir l'URL publique.

50 min-1 h 50 — Pratique

1. L'apprenant pousse son app sur un repo GitHub (privé).
2. Il importe le projet dans Vercel.
3. Il reporte ses variables d'environnement (Supabase, etc.) dans les réglages Vercel (jamais dans le code).
4. Il déclenche le déploiement, puis ouvre son app sur l'URL publique.
5. Il teste le parcours complet en ligne : inscription, connexion, données.
6. PROGRESS.md + DECISIONS.md + commit.

1 h 50-2 h — Récap + checkpoint + devoir

- Moment fort : son app est EN LIGNE. Lui faire envoyer l'URL à un proche pour tester.
- **Devoir** : faire tester l'app par 2 personnes et noter leurs retours et bugs.

Points de vigilance

- **Variables d'env** : la bêtise classique = oublier de les mettre dans Vercel → app cassée en prod. Vérifier ensemble.
- Supabase : s'assurer que les URL autorisées (redirect auth) incluent le domaine Vercel.
- Ne pas committer le `.env` (le hook protège, mais vérifier).
- **Économie Pro** : le déploiement Vercel ne consomme pas de quota Claude ; les corrections oui.

Démo possible sur un vrai projet

Montrer une de tes apps en prod et son pipeline de déploiement.

Critère de réussite

L'app est accessible par une URL publique et le parcours complet fonctionne en ligne.

S16

Tester la sécurité de sa propre app + démo finale

Objectif : l'apprenant audite la sécurité de SA propre app déployée (test autorisé), corrige les failles trouvées, puis fait la démo finale et explique sa méthode.

Cadre éthique (à poser d'entrée) : on ne teste QUE sa propre app, qu'on a le droit de tester. Tester l'app d'autrui sans autorisation écrite est illégal. Cette règle est non négociable.

Déroulé (2 h)

0-10 min — Retour devoir + cadre

- Retours des 2 testeurs : on note les bugs.
- Poser le cadre éthique du pentest : **autorisation obligatoire, uniquement sa cible**.

10-35 min — Concept : penser comme un attaquant (sur sa propre app)

- Pourquoi : une app avec login + base est une cible. Mieux vaut trouver les failles soi-même.
- Les classiques à vérifier sur une app web :
- Un utilisateur peut-il voir les données d'un autre ? (contrôle d'accès, le plus fréquent)
- Les pages protégées le sont-elles vraiment (pas juste cachées) ?
- Les entrées sont-elles validées (formulaires) ?
- Les clés/secrets ne fuient-ils pas côté client ?
- Le Jarvis a des agents/skills pour ça : `security-auditor` (défensif), `web-hunter`, `api-security`, `bizlogic-hunter`, skill `webapp-exploit-hunter`. On les utilise sur SA cible.
- Démo : lancer `security-auditor` sur son app, lire les findings.

35-50 min — Démo : un test de contrôle d'accès

- Montrer comment vérifier qu'un utilisateur A ne peut pas accéder aux données de B (le bug le plus courant d'une app débutante avec Supabase mal configuré / RLS absent).

50 min-1 h 35 — Pratique (audit + correction)

1. L'apprenant lance un audit de sécurité sur son app (agent `security-auditor` d'abord, défensif).
2. Il teste manuellement le contrôle d'accès (2 comptes, vérifier l'isolation des données).
3. Il vérifie que ses secrets ne sont pas exposés côté client.
4. Il corrige les failles trouvées (souvent : activer/corriger les règles RLS Supabase, protéger une route).
5. Il redéploie et revérifie.
6. PROGRESS.md + DECISIONS.md + commit.

1 h 35-2 h — Démo finale + bilan

- L'apprenant fait une **démo de 10 min** : il montre son app en ligne, le parcours complet, et explique SA méthode de travail (spec, continuité, MCP, agents, hooks, déploiement, sécurité).
- Bilan : ce qu'il sait faire seul maintenant, ce qu'il veut continuer à creuser.
- Lui rappeler : il repart avec son Jarvis, il peut lancer un nouveau projet quand il veut avec `/nouveau-projet`.

Points de vigilance

- **Éthique avant tout** : marteler que ces outils ne servent QUE sur ses propres cibles autorisées. Un mésusage a des conséquences légales.
- Rester sur du test applicatif de SA cible, pas d'incursion sur des infrastructures tierces.
- La faille la plus probable de son app : données non isolées entre utilisateurs (RLS Supabase). Bien vérifier.
- **Économie Pro** : un audit complet consomme ; cibler les points sensibles (auth, accès données).

Démo possible sur un vrai projet

Montrer un exemple de faille d'accès que tu as corrigée sur un de tes projets (sans exposer de vraie donnée).

Critère de réussite

L'app a été auditée, au moins la faille d'isolation des données a été vérifiée et corrigée, et l'apprenant fait une démo autonome en expliquant sa méthode.